

Maximum Likelihood Estimation

Where every loss function comes from

Prof. Nipun Batra

Mathematical Foundations for AI · IIT Gandhinagar

The hook: who chose your loss function?

Three lines you will type for the rest of your career

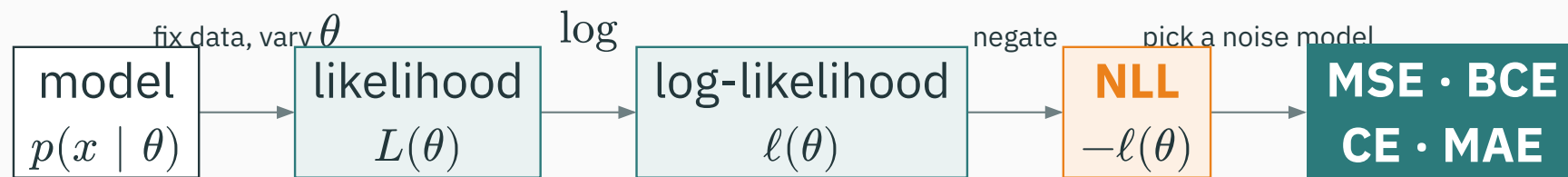
Every ML training loop — every paper, every framework — ends the same way:

```
loss = F.mse_loss(y_hat, y)           # regression
loss = F.binary_cross_entropy(p, y)    # yes/no classification
loss = F.cross_entropy(logits, y)      # multi-class classification
```

Squaring errors. Taking logs of probabilities. **Who decided these formulas?** Why squared — not absolute, not fourth power? Why is there a logarithm inside a classification loss?

Nobody tuned these by trial and error. All three fall out of **one idea**, mechanically. Today: the idea, and the full derivation for MSE. Cross-entropy's turn comes in L24 — but you'll leave today knowing exactly where it lives.

Today's one move, as a map



One move — **flip the model around** — then bookkeeping. Every box on this map gets built today, with numbers.

You have never formally “fit a model” in this course. That is the point: today is where **fitting** becomes a mathematical operation instead of a vibe.

Learning outcomes

By the end of this lecture you will be able to:

1. Read $p(x \mid \theta)$ **two ways**: as a distribution over x , and as a likelihood over θ .
2. Write the likelihood of an entire dataset using the **IID assumption**.
3. Explain **three independent reasons** the log goes in — and recap L2's 0.03^{1000} disaster.
4. Derive $\hat{\theta}_{\text{MLE}}$ for the **Bernoulli** coin (★) and the **Gaussian** mean.
5. Derive the course's keystone: **minimizing MSE = maximizing Gaussian likelihood** (★).
6. Map noise models to the losses they generate: Gaussian→MSE, Bernoulli→BCE, Categorical→CE, Laplace→MAE.

The flip: from probability to likelihood

Pop quiz · one student, three courses

Three courses grade on curves. Their score distributions:

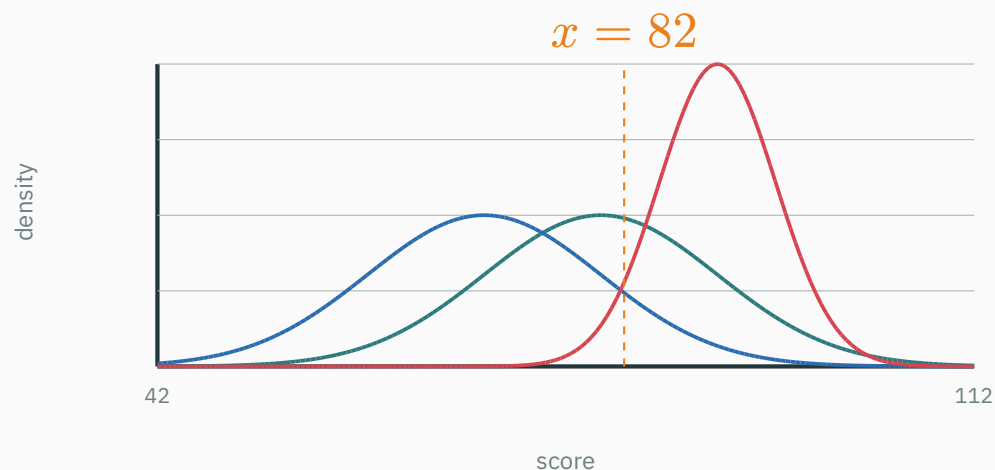
- C1: $\mathcal{N}(\mu = 80, \sigma = 10)$
- C2: $\mathcal{N}(\mu = 70, \sigma = 10)$
- C3: $\mathcal{N}(\mu = 90, \sigma = 5)$

I stop a random student in the corridor. They took exactly one of the three, and they scored **82**.

Which course would you guess? Commit to an answer — and notice what your brain just did to produce it.

The guess, made honest

— c1: $\mathcal{N}(80, 10^2)$ — c2: $\mathcal{N}(70, 10^2)$ — c3: $\mathcal{N}(90, 5^2)$



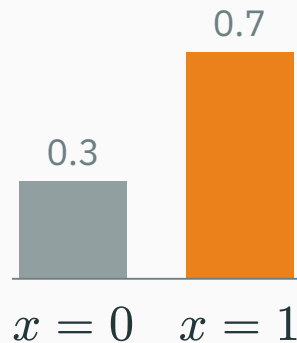
Read each curve **at the observed score**: $p(82 | C1) = 0.0391$, $p(82 | C2) = 0.0194$, $p(82 | C3) = 0.0222$.

C1 wins: you picked the model under which the data is **least surprising.
That's maximum likelihood — you already do it by eye.**

The flip, on the simplest model there is

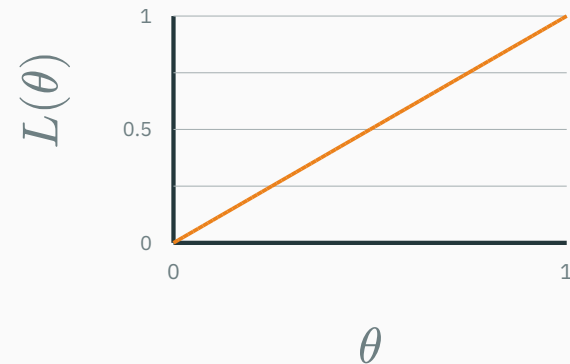
A coin with $P(\text{heads}) = \theta$. One formula, $p(x \mid \theta)$ — **two completely different readings**:

Fix $\theta = 0.7$, vary x



a **distribution** over outcomes
bars sum to 1 — guaranteed

Fix $x = 1$ (one head), vary θ



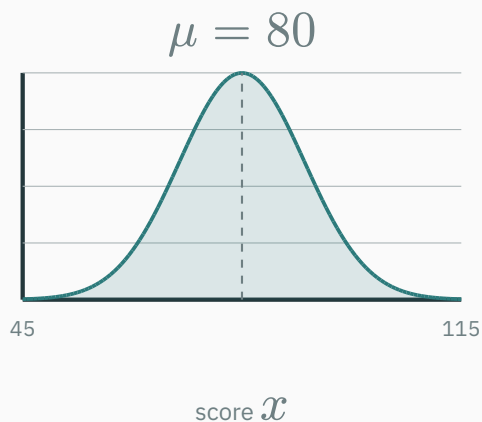
the **likelihood** of that outcome
area under this line: $1/2$ — no promise at all

Two bars became a line: **different axis, different object** — same formula.

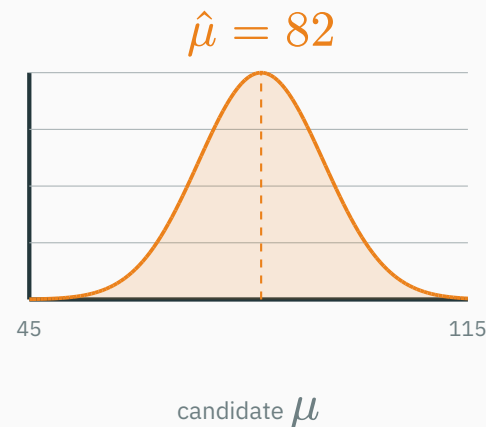
The flip, on the corridor student

Now suppose we know the course has $\sigma = 10$ but **nobody told us μ** . The student says 82.

DISTRIBUTION: fix $\mu = 80$, vary x



LIKELIHOOD: fix $x = 82$, vary μ



The right curve peaks at $\mu = 82$: the single best explanation centers the bell **on the data**. You just estimated a parameter.

Naming the move

$L(\theta) := p(\mathcal{D} \mid \theta)$, read as a function of θ with the data $\mathcal{D}$ **frozen**.

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} L(\theta).$$

- $p(x \mid \theta)$ as a function of x : a **distribution** — normalized, sums/integrates to 1
- the same formula as a function of θ : the **likelihood** — a plain score, one number per candidate model

$L(\theta)$ is **not** a probability distribution over θ : nothing makes its area 1 (the one-head line had area 1/2). “Which θ is most probable” is a different, stronger question — that’s Bayes and L15. Today we only ask which θ **explains the data best**.

The likelihood of a dataset

From one observation to a dataset

One data point gives one score: $L(\theta) = p(x_1 \mid \theta)$. A dataset $\mathcal{D} = \{x_1, \dots, x_n\}$ needs the **joint** probability $p(x_1, \dots, x_n \mid \theta)$ — and joints are hard. We buy simplicity with an assumption. **IID** is two separate claims:

- **Independent** — given θ , no observation tells you about another $\rightarrow$ the joint **factors into a product**
- **Identically distributed** — every observation uses the **same** $p(\cdot \mid \theta)$, one shared θ

$$L(\theta) = \prod_{i=1}^n p(x_i \mid \theta)$$

Each claim can fail alone: tomorrow's weather depends on today's ($\rightarrow$ L25's Markov chains); two different-noise sensors share no rule. IID is a modeling **choice** — say it out loud.

IID at work · two coins, two tiny datasets

Candidate coins: fair ($\theta = 0.5$) vs head-heavy ($\theta = 0.8$). Score both on data:

Observed	$L(0.5)$	$L(0.8)$	Better explanation
H, T	$0.5 \times 0.5 = 0.25$	$0.8 \times 0.2 = 0.16$	the fair coin
H, H	$0.5^2 = 0.25$	$0.8^2 = 0.64$	the head-heavy coin

Same two candidates, different frozen data → different winner. The likelihood **ranks models by the data they actually saw**.

Independence licenses the multiplication; identical distribution licenses reusing one θ in every factor.

Pop quiz, round 2 · now with a dataset

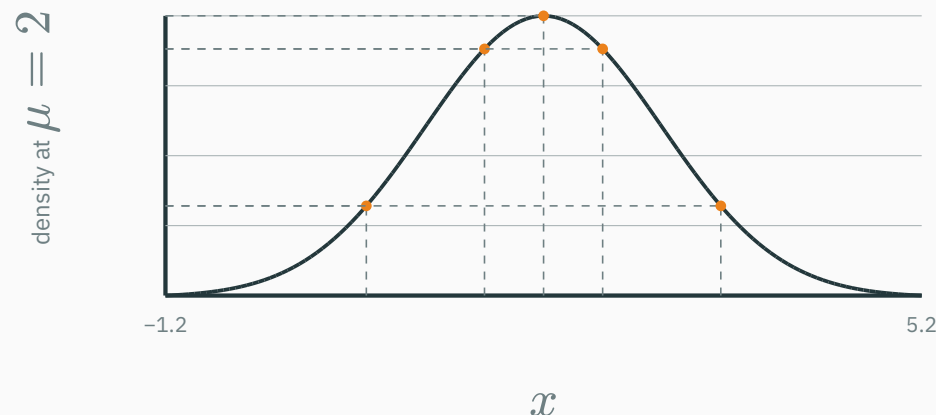
Back in the corridor. **Two** students this time — scores 82 and 72 — and both took the **same** (unknown) course. Multiply each course's two density readings:

Course	$p(82 \mid \cdot)$	$p(72 \mid \cdot)$	$L = \text{product}$
C1: $\mathcal{N}(80, 10^2)$	0.0391	0.029	1.1×10^{-3}
C2: $\mathcal{N}(70, 10^2)$	0.0194	0.0391	7.6×10^{-4}
C3: $\mathcal{N}(90, 5^2)$	0.0222	0.0001	2.7×10^{-6}

C1 wins again — but look at C3: **one** far-off point (72, at $z = -3.6$) collapsed its entire product. A product is a chain of vetoes: every point must be plausible.

A dataset's likelihood multiplies one reading per point — computed with the IID recipe you just learned.

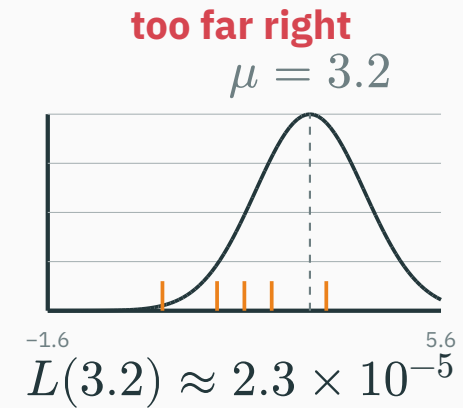
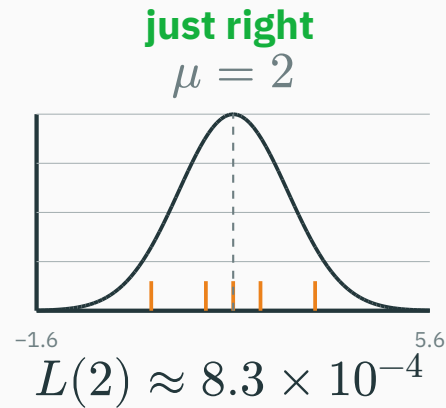
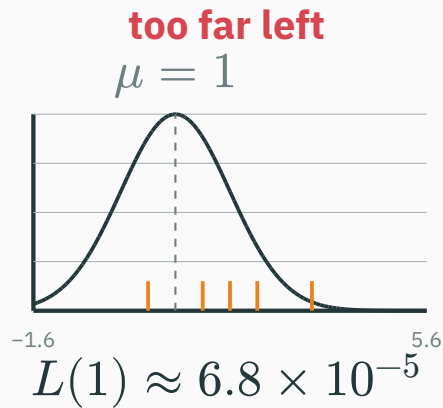
Our running dataset: $\mathcal{D} = \{0.5, 1.5, 2.0, 2.5, 3.5\}$, modeled as $\mathcal{N}(\mu, 1)$ with σ known. Try $\mu = 2$: read the density at each point, multiply.



$$L(2) = 0.13 \times 0.352 \times 0.399 \times 0.352 \times 0.13 \approx 8.3 \times 10^{-4}$$

One number for the candidate $\mu = 2$. Now the key mental animation: **slide the bell and watch the score move.**

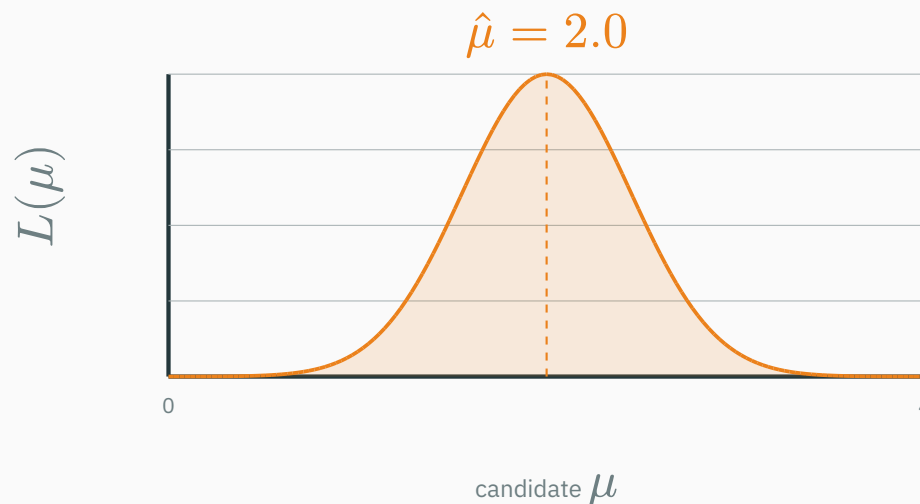
Slide the Gaussian over the data



Same five orange data points every time; only the model slides. Centered wins by a factor of **12×** over $\mu = 1$.

Fitting a model = sliding its parameters until the data's likelihood is maximal.

Evaluate $L(\mu)$ at **every** candidate μ — the likelihood function of this dataset:



- Each point of this curve is one “slide position” from the previous slide — a **product of five densities**
- The peak sits at $\hat{\mu} = 2.0$ — which happens to be the sample mean of $\mathcal{D}$. Coincidence? We’ll **prove** it shortly.

The curve $L(\mu)$ we just drew for $\mathcal{D} = \{0.5, 1.5, 2.0, 2.5, 3.5\}$ — which statement is true?

A. It is a probability density over μ , so its area must be 1

B. $L(2.0)$ is the probability that $\mu = 2.0$ is the true parameter

C. $L(\mu)$ is the joint density of the five **observed** points, re-read as a function of μ

D. Its peak location depends on the order in which the five points were multiplied

Answer: what kind of object is $L(\mu)$?

A ANSWER

Correct: C — the joint density of the observed data, as a function of μ

Why: The data is frozen; μ varies. Nothing normalizes the curve over μ (A), and no probability is ever assigned **to** μ — that requires a prior and Bayes' rule, which is L15's business (B). Multiplication is commutative, so order is irrelevant (D).

**Log-likelihood: products
become sums**

Products of probabilities are numerically doomed

You proved this in L2 — the recap: a language model scores a 1000-character text, each character with probability ≈ 0.03 :

$$P(\text{text}) = \prod_{i=1}^{1000} p_i \approx 0.03^{1000} \approx 10^{-1523} \Rightarrow \underset{\text{even float64}}{0.0} \quad \text{argmax of all-zeros} = \text{garbage}$$

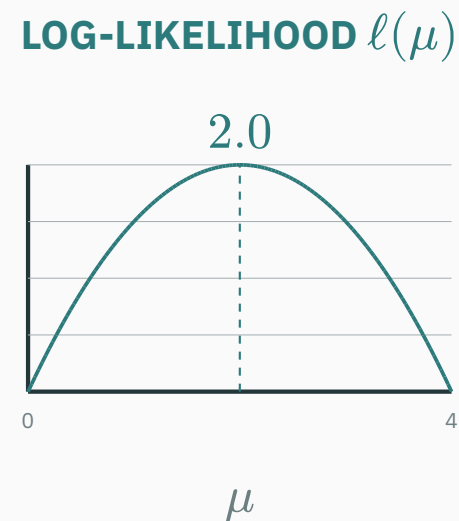
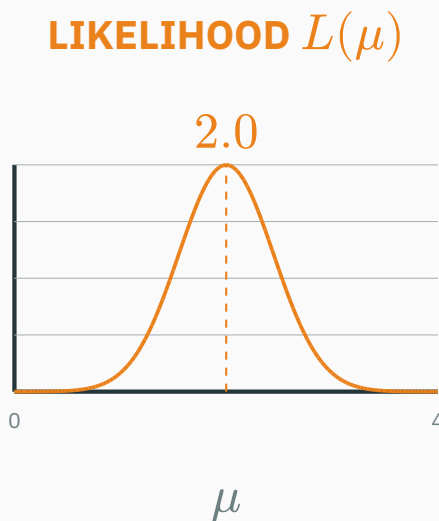
Our five-point likelihood already sat at 10^{-4} . In log-space, tiny becomes ordinary:

$$\log P(\text{text}) = \sum_{i=1}^{1000} \log p_i \approx 1000 \times (-3.51) = -3507 \quad \checkmark \text{ a boring, safe float}$$

L2's exact words: “Every loss you will ever meet is a *log*-likelihood — floating point demands it (**L14 shows the deeper reason**).” Today is that day — and underflow is only reason 1 of 3.

Reason 2 · the log moves the peak nowhere

log is **strictly increasing**: $a > b > 0 \Rightarrow \log a > \log b$. So it preserves every comparison of likelihoods — including where the maximum sits:



$$\arg \max_{\theta} L(\theta) = \arg \max_{\theta} \log L(\theta)$$

— same peak, tamer numbers

Reason 3 · sums are what calculus (and autodiff) want

$$\ell(\theta) := \log L(\theta) = \log \prod_{i=1}^n p(x_i \mid \theta) = \sum_{i=1}^n \log p(x_i \mid \theta)$$

To maximize, we differentiate (L7–L8) and set to zero. Compare the two forms:

- $\frac{d}{d\theta} \prod_i p_i$: the product rule over n factors — n terms, each a product of $n - 1$ densities
- $\frac{d}{d\theta} \sum_i \log p_i = \sum_i \left(\frac{d}{d\theta} \right) \log p_i$: differentiate each term **alone**, add

Three independent wins — no underflow (L2), same argmax, per-point derivatives. This is why **every training objective you will ever see is built from $\log p$.**

Flip the sign, and you have invented “the loss”

Optimizers by convention **minimize** (Module 4 is built that way). So negate:

$$\text{NLL}(\theta) = -\ell(\theta) = -\sum_{i=1}^n \log p(x_i \mid \theta) \quad \text{and often the mean} \quad \frac{1}{n} \sum_i -\log p(x_i \mid \theta)$$

- maximize likelihood $\Leftrightarrow$ maximize log-likelihood $\Leftrightarrow$ **minimize NLL**
- each data point contributes $-\log p(x_i \mid \theta)$: **its personal penalty** — large when the model found it implausible

A “loss” is not a design flourish. Training loss = per-example negative log-likelihood, averaged. This sentence is the hinge of the whole course.

**The coin: your first MLE,
derived **

The model, and a clever way to write it

Flip a coin 10 times: $H, H, T, H, H, H, T, H, H, T$ — **7 heads, 3 tails**. Model: each flip is Bernoulli, $P(H) = \theta$, IID. Code heads as $x = 1$, tails as $x = 0$:

$$p(x \mid \theta) = \theta^x (1 - \theta)^{1-x} \quad \text{one formula, both cases: } x = 1 \rightarrow \theta; x = 0 \rightarrow 1 - \theta$$

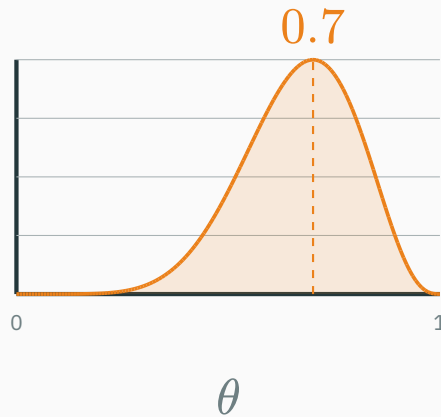
The dataset's likelihood — products of θ 's and $(1 - \theta)$'s, one per flip:

$$L(\theta) = \prod_{i=1}^{10} \theta^{x_i} (1 - \theta)^{1-x_i} = \theta^7 (1 - \theta)^3$$

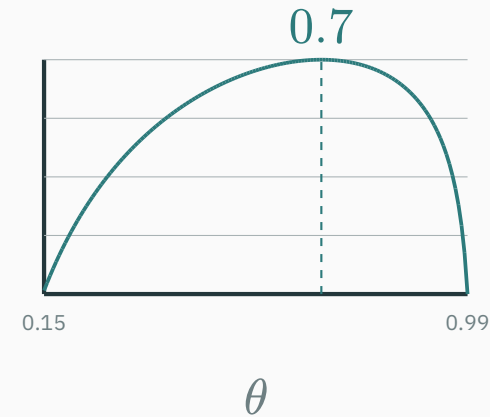
Only the **counts** survive: 7 heads, 3 tails. The order of flips never mattered — IID told us so before we computed anything.

The two curves, computed

$$L(\theta) = \theta^7 (1 - \theta)^3$$



$$\ell(\theta) = 7 \log \theta + 3 \log(1 - \theta)$$



Both peak at $\theta = 0.7$. Spot values: $L(0.5) = 0.5^{10} \approx 9.8 \times 10^{-4}$, $L(0.7) \approx 2.2 \times 10^{-3}$ — the data is 2.3× more plausible under $\theta = 0.7$ than under a fair coin.

★ Derivation · set the derivative to zero

Maximize $\ell(\theta) = 7 \log \theta + 3 \log(1 - \theta)$ the L7 way — stationary point:

$$\frac{d\ell}{d\theta} = \frac{7}{\theta} - \frac{3}{1-\theta} = 0$$

$$7(1 - \theta) = 3\theta \Rightarrow 7 = 10\theta \Rightarrow \hat{\theta} = \frac{7}{10} = 0.7$$

Is it a maximum? Check curvature (L7's second-derivative test):

$$\frac{d^2\ell}{d\theta^2} = -\frac{7}{\theta^2} - \frac{3}{(1-\theta)^2} < 0 \quad \text{everywhere on } (0, 1)$$

Strictly concave $\rightarrow$ the single stationary point is the **global** maximum. No luck involved.

★ Derivation · now for any dataset

General case: n flips, n_H heads, $n_T = n - n_H$ tails. Same three moves:

$$\ell(\theta) = \sum_{i=1}^n [x_i \log \theta + (1 - x_i) \log(1 - \theta)] = n_H \log \theta + n_T \log(1 - \theta)$$

$$\frac{d\ell}{d\theta} = \frac{n_H}{\theta} - \frac{n_T}{1 - \theta} = 0 \quad \Rightarrow \quad n_H(1 - \theta) = n_T\theta$$

$$n_H = (n_H + n_T)\theta$$

$$\hat{\theta}_{\text{MLE}} = \frac{n_H}{n} \text{ — the fraction of heads.}$$

The answer your intuition would have guessed — but now it is a **theorem**, from a principle that will still work when intuition has nothing to say (a billion-parameter model has no “fraction of heads”).

Sanity checks, one honest and one alarming

The machine agrees — brute-force the curve:

```
heads, n = 7, 10
theta = np.linspace(0.01, 0.99, 981)
loglik = heads * np.log(theta) + (n - heads) * np.log(1 - theta)
theta[np.argmax(loglik)]          # 0.7 - the head fraction
```

Now flip a coin **3 times**, get **3 heads**: $\hat{\theta}_{\text{MLE}} = 3/3 = 1.0$ — this coin **cannot ever** land tails. Three flips convinced MLE of a certainty. Your intuition says “probably just a normal coin”. Your intuition is using a **prior** — and giving MLE one is exactly L15.

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/mle-map-coin

The **mle-map-coin** article flips coins live: the likelihood curve rebuilds after every flip, and you drag θ to feel the score change under your cursor. Recreate today's 7-of-10, then try 3-of-3 and watch the peak slam into the boundary at $\theta = 1$. (Ignore the prior slider for now — after L15 it will be the whole point.)

Two-minute experiment before next lecture: keep the head **fraction** at 70% but grow n : 7/10 $\rightarrow$ 70/100 $\rightarrow$ 700/1000. Watch what happens to the curve's **width** — we name that phenomenon at the end of today.

MLE for the Gaussian

The Gaussian log-likelihood, simplified once and for all

Model $\mathcal{D} = \{x_1, \dots, x_n\}$ as IID $\mathcal{N}(\mu, \sigma^2)$ (L12's density, L13's star):

$$\ell(\mu, \sigma) = \sum_{i=1}^n \log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{(x_i - \mu)^2}{2\sigma^2} \right) \right]$$

log of (product of exp) — everything unfolds:

$$\ell(\mu, \sigma) = -\frac{n}{2} \log(2\pi) - n \log \sigma - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

All the μ -dependence sits in one term: $-\sum_i (x_i - \mu)^2$. Maximizing ℓ over μ is minimizing a sum of squares. Remember this shape — it returns in ten minutes as MSE.

The mean, earned

Differentiate w.r.t. μ (the constants and $-n \log \sigma$ vanish):

$$\frac{\partial \ell}{\partial \mu} = \frac{1}{\sigma^2} \sum_{i=1}^n (x_i - \mu) = 0 \quad \Rightarrow \quad \sum_i x_i = n\mu$$

$$\hat{\mu}_{\text{MLE}} = \frac{1}{n} \sum_{i=1}^n x_i = \bar{x}$$

On our running dataset: $\hat{\mu} = (0.5 + 1.5 + 2.0 + 2.5 + 3.5)/5 = 2.0$ — **exactly where the likelihood curve peaked** in Section 3. The picture and the calculus agree.

Note what $\hat{\mu}$ does **not** depend on: σ . However noisy the ruler, the best center is the average. (With σ unknown, the μ -equation is unchanged.)

The variance — and an honest footnote

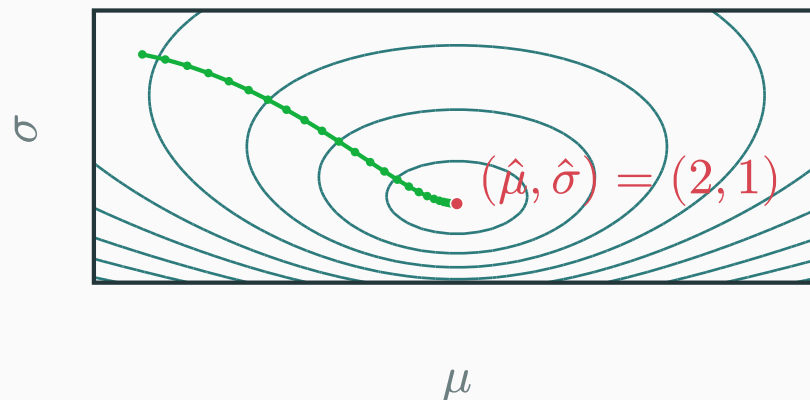
Same recipe for σ : differentiate $\ell = \dots - n \log \sigma - \frac{1}{2\sigma^2} \sum_i (x_i - \mu)^2$, set to zero, solve:

$$\frac{\partial \ell}{\partial \sigma} = -\frac{n}{\sigma} + \frac{1}{\sigma^3} \sum_i (x_i - \hat{\mu})^2 = 0 \quad \Rightarrow \quad \hat{\sigma}_{\text{MLE}}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

On $\mathcal{D}$: deviations $(-1.5, -0.5, 0, 0.5, 1.5)$, squares sum to 5.0, so $\hat{\sigma}^2 = 5.0/5 = 1.0$.

Honest footnote. This estimator is **biased**: on average across datasets, $\mathbb{E}[\hat{\sigma}^2] = \frac{n-1}{n} \cdot \sigma^2$ — systematically a bit small, because $\bar{x}$ chased the sample before we measured spread around it. NumPy's `np.var(x, ddof=1)` applies the $\frac{n}{n-1}$ repair (Bessel's correction). We state this; a statistics course proves it. MLE is **excellent**, not sacred.

Free both parameters: the **dataset NLL** over the (μ, σ) plane, computed live from $\mathcal{D}$



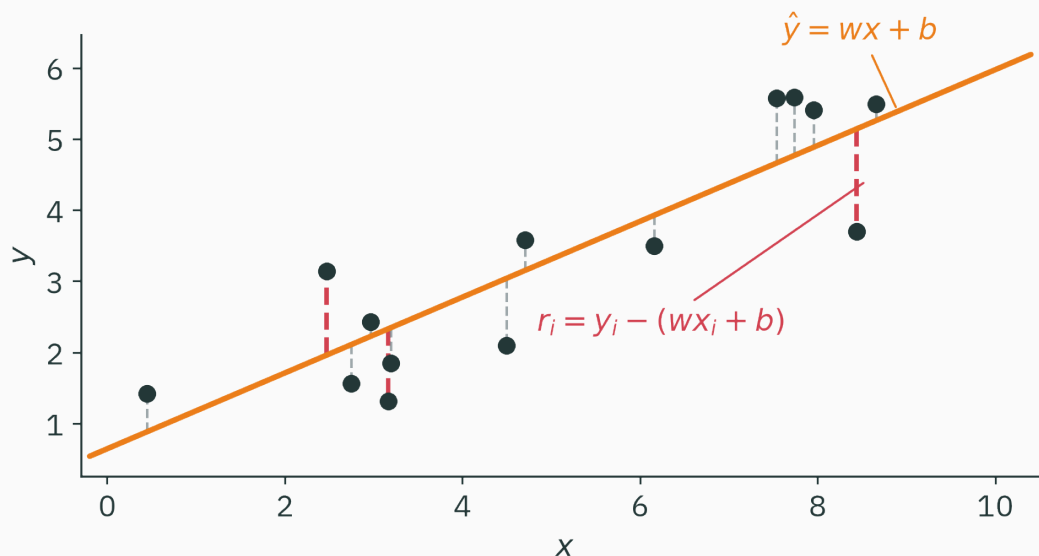
- The bowl bottoms out at $(\hat{\mu}, \hat{\sigma}) = (2.0, 1.0)$ — both closed forms at once
- The green path: **gradient descent** walking downhill on this exact surface

Fitting = minimizing NLL. Module 4 (L16–L21) is the art of **walking downhill on such surfaces.**

The keystone: linear regression
→ MSE

A model that finally has an input

Everything so far fit a **constant** distribution. Real problems predict y **from** x — rent from area, temperature from hour. The simplest such model is a line:

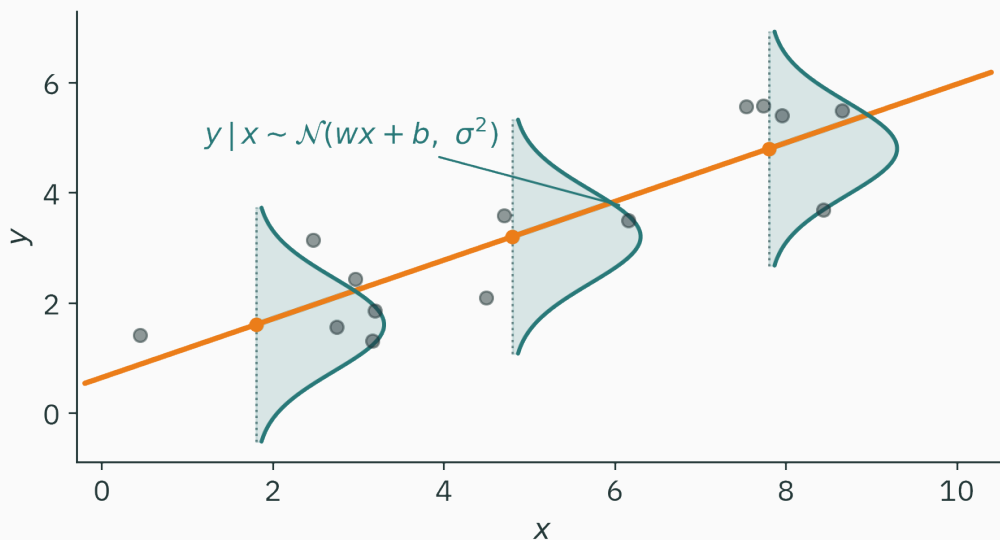


$\hat{y} = wx + b$; the **residual** $r_i = y_i - (wx_i + b)$ is what the line cannot explain. Which line is **best**? We need a likelihood — so we need a **distribution over** y .

The noise model: a bell standing on the line

Assume the world draws y as **the line plus Gaussian noise**:

$$y_i = wx_i + b + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2) \text{ IID} \quad \Rightarrow \quad y_i \mid x_i \sim \mathcal{N}(wx_i + b, \sigma^2)$$



A vertical Gaussian stands **on the line** at every x : near the line, high density; stragglers, expensive. That is the whole assumption — now we turn the crank.

The likelihood of a **labeled** dataset

New situation, same machinery. The data comes in pairs (x_i, y_i) , and our model only says how y arises **given** x — it never models the inputs:

$$L(\theta) = \prod_{i=1}^n p(y_i \mid x_i, \theta) \quad \theta = (w, b); \text{ the } x_i \text{ enter as fixed givens}$$

- Same flip: data (x_i, y_i) frozen, parameters (w, b) varying
- Same IID product — independence **of the noise** ε_i , one shared line
- No $p(x)$ anywhere: whoever chose the inputs is not our problem

This conditional form is the shape of **every** supervised objective — including the trillion-token ones: L26's language model maximizes exactly such a product, one conditional per next character.

★ Derivation · one point's log-likelihood

The density of observing label y_i at input x_i , given parameters $\theta = (w, b)$:

$$p(y_i \mid x_i, \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - (wx_i + b))^2}{2\sigma^2}\right)$$

Take the log — the same unfold as the Gaussian section:

$$\log p(y_i \mid x_i, \theta) = -\frac{(y_i - (wx_i + b))^2}{2\sigma^2} - \log \sigma - \frac{1}{2} \log(2\pi)$$

$$-\log p(y_i \mid x_i, \theta) = \underbrace{\frac{(y_i - \hat{y}_i)^2}{2\sigma^2}}_{\text{squared residual!}} + \underbrace{\log \sigma + \frac{1}{2} \log(2\pi)}_{\text{no } \theta \text{ inside — constant}}$$

One data point's NLL **is** its squared residual, rescaled and shifted by constants.

Sum over the IID dataset and maximize over $\theta = (w, b)$, with σ fixed:

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} \ell(\theta) = \arg \min_{\theta} \sum_{i=1}^n \left[\frac{(y_i - \hat{y}_i)^2}{2\sigma^2} + \log \sigma + \frac{1}{2} \log(2\pi) \right]$$

Adding a constant never moves an argmin — drop $n \log \sigma + \frac{n}{2} \log(2\pi)$. Scaling by a positive constant never moves an argmin — drop $1/(2\sigma^2)$:

$$\hat{\theta}_{\text{MLE}} = \arg \min_{w,b} \sum_{i=1}^n (y_i - wx_i - b)^2 \text{ — Gaussian MLE is least squares.}$$

Nobody “invented” MSE: it was hiding inside $\mathcal{N}$ all along.

What MSE silently assumes

Run the derivation backwards: using MSE **is** declaring a noise model. Three claims, made every time you call `mse_loss`:

1. noise is **Gaussian** — bell-shaped errors around the prediction
2. noise has **the same σ everywhere** — no input is harder than another
3. observations are **independent** given the inputs

Assumption 1 is how outliers wreck least squares: a point 5σ off the line pays $(5\sigma)^2/2\sigma^2 = 12.5$ nats — the Gaussian is **certain** such points are freakish, so the fitted line contorts to appease them. Heavier-tailed noise models forgive them — that's the MAE row coming up.

The machine agrees · one grid, two objectives

```
x = np.array([0., 1., 2., 3., 4.]); y = np.array([1.1, 1.9, 2.8, 4.3, 4.9])
ws = np.linspace(0.2, 1.8, 801) # candidate slopes (b = 1 fixed)
mse = [np.mean((y - w*x - 1.0)**2) for w in ws]
ll = [np.sum(-0.5*np.log(2*np.pi) - 0.5*(y - w*x - 1.0)**2) for w in ws]
print(ws[np.argmin(mse)], ws[np.argmax(ll)]) # 1.0 1.0 - the same slope
```

Two objectives, one winner — on every dataset, always, by the derivation you just did.

This is why earlier stats courses could teach least squares without ever saying “likelihood”: with σ fixed, the two problems are **literally the same problem**. You now know which one is fundamental.

Every loss is an NLL

The table this course orbits around

Choose a noise/observation model; the NLL **is** your loss:

You observe	Model	NLL per example	Its famous name
a real y	Gaussian	$(y - \hat{y})^2 / 2\sigma^2 + c$	MSE ✓ today
yes / no	Bernoulli	$-[y \log p + (1 - y) \log(1 - p)]$	BCE ✓ today
1 of K classes	Categorical	$-\log p_{\text{true class}}$	cross-entropy → L24
y with outliers	Laplace	$ y - \hat{y} / b + c$	MAE bonus

Loss functions are not a menu of arbitrary formulas. Each row is the same construction — only the noise model changes.

The BCE row costs one line

For a yes/no label $y \in \{0, 1\}$ with predicted head-probability p , the Bernoulli pmf in exponent form (from the coin section!) gives the per-example NLL:

$$-\log[p^y(1-p)^{1-y}] = -[y \log p + (1-y) \log(1-p)]$$

That **is** `binary_cross_entropy` — the second line of the hook’s code. You derived it as the coin’s log-likelihood before you knew its name.

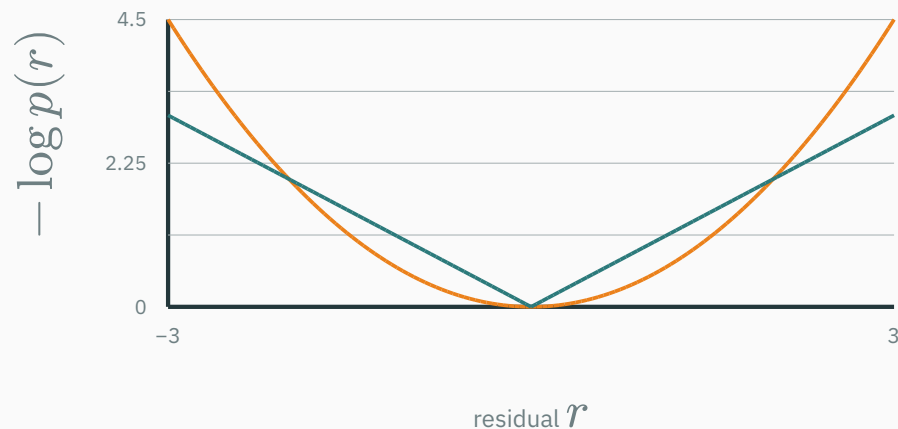
- $y = 1$, model said $p = 0.9$: penalty $-\log 0.9 \approx 0.105$ — cheap
- $y = 1$, model said $p = 0.01$: penalty $-\log 0.01 \approx 4.6$ — confident wrongness is **expensive**, by the shape of $-\log$

The categorical / cross-entropy row is the same move over K classes — L24 does it properly and connects it to information theory (why “entropy” is in the name), and L2’s stable log-softmax is exactly how it’s computed.

Two noise models, two loss shapes

The **true** per-residual loss curves, computed from the densities themselves ($-\log p$, shifted to 0 at $r = 0$):

— Gaussian $\rightarrow r^2/2$ (MSE-shaped) — Laplace $\rightarrow |r|$ (MAE-shaped)



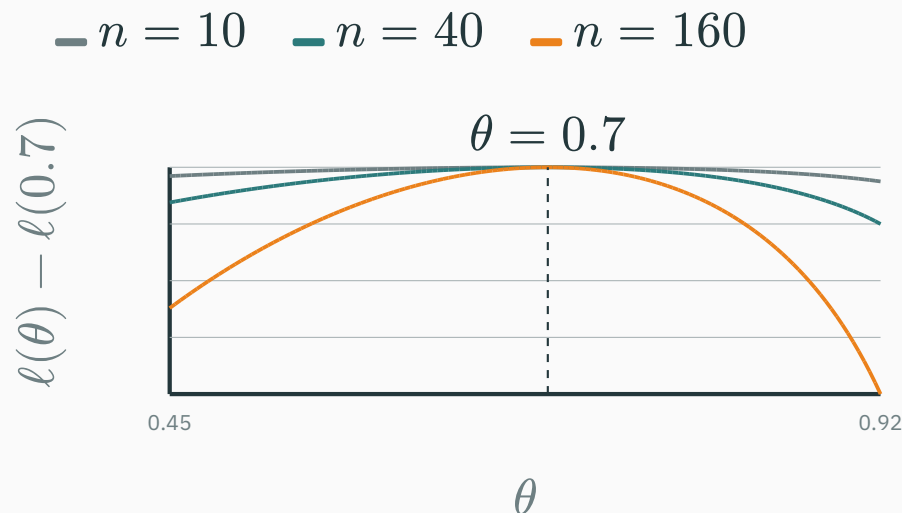
The parabola **accelerates**: doubling a large residual quadruples its cost — outliers steer the fit. The Laplace's thin tails in log-space charge a flat rate — outliers get a shrug. Choosing a loss **is** choosing how much you believe in outliers.

The hook, revisited · case status

Loss	Where it comes from	Status
mse_loss	Gaussian noise on the prediction — derived, boxed, yours	closed
binary_cross_entropy	Bernoulli NLL — the coin, renamed	closed
cross_entropy	Categorical NLL + information theory: why it's called entropy , what it has to do with compression	open → L24

L2 told you `CrossEntropyLoss` fuses log-softmax for stability and takes raw logits. Today you learned why it's a **loss** at all. L24 explains why it's called **entropy**. Three lectures, one function — that's how central it is.

Hold the head fraction at 70%, grow n — the peak-aligned log-likelihood sharpens:



- **Consistency** (stated): as $n \rightarrow \infty$, $\hat{\theta}_{\text{MLE}} \rightarrow$ the true θ — alternatives get buried
- Small n misbehaves: biased $\hat{\sigma}^2$, certainty from 3 flips. Data cures MLE; priors (L15) cure small data.

Your sensor's errors are usually small but show occasional huge spikes — well modeled by Laplace noise. Maximum likelihood hands you which training loss?

A. MSE — squared error is always optimal

C. BCE — it has a log, logs handle spikes

B. MAE — the mean **absolute** error

D. Cross-entropy over discretized bins

Correct: B — MAE — mean absolute error

Why: Laplace density $\propto e^{-|r|/b}$, so per-point NLL = $|r|/b + c$: the absolute residual. The quadratic MSE would let each spike bully the fit (r^2 explodes); the linear penalty stays calm. Same construction as MSE — different noise belief. That is the table doing real work.

Practice problems

Try on paper; the T7 notebook (after L15) checks all of them in NumPy.

1. 12 flips, 9 heads: write $\ell(\theta)$, maximize it, and compute $\ell(0.75) - \ell(0.5)$.
2. For $\mathcal{D} = \{1.0, 3.0\}$ under $\mathcal{N}(\mu, 1)$: sketch $L(\mu)$; find $\hat{\mu}$, then $\hat{\sigma}^2$. (Answers: 2.0 and 1.0.)
3. Exponential waiting times $p(x \mid \lambda) = \lambda e^{-\lambda x}$ (L12's zoo): derive $\hat{\lambda}_{\text{MLE}} = 1/\bar{x}$.
4. Redo the keystone with **Laplace** noise and show the objective becomes $\sum_i |y_i - wx_i - b|$.
5. For $n = 10^6$, $p_i \approx 0.1$: estimate $\prod_i p_i$ and compare with float64's smallest positive number ($\approx 10^{-324}$, L2). Why must training code never form this product?
6. True or false, one sentence: “the MLE of θ is the most probable value of θ given the data.” (The L15 trap.)

Lecture 14 — summary

- **The flip**: freeze the data, vary θ — the likelihood is a score, not a distribution over θ .
- **Datasets**: IID $\rightarrow$ a product of per-point densities; slide the model, climb the score.
- **Logs**: same argmax, no underflow (L2's 0.03^{1000} , cached), per-point derivatives. NLL = **the loss**.
- **Coin** ★: $\hat{\theta} = n_H/n$ from $\ell'(\theta) = 0$; 3-for-3 heads exposes MLE's blind spot ($\rightarrow$ L15).
- **Gaussian**: $\hat{\mu} = \bar{x}$; $\hat{\sigma}^2 = \frac{1}{n} \sum (x_i - \bar{x})^2$ — biased, honestly footnoted.
- **Keystone** ★: Gaussian noise on a line $\rightarrow$ max likelihood $\equiv$ min squared error.
- **The table**: Gaussian $\rightarrow$ MSE, Bernoulli $\rightarrow$ BCE, Categorical $\rightarrow$ CE (L24), Laplace $\rightarrow$ MAE.

Read before L15 — MML §8.1–8.3 (skim 8.1–8.2, focus 8.3) · MacKay Ch. 2. **T7** (after L15) drills MLE + MAP together.



Where MLE points as $n \rightarrow \infty$

★ OPTIONAL

Divide the log-likelihood by n : an **average** that (by large numbers) approaches an expectation over the true data source p^* :

$$\frac{1}{n}\ell(\theta) = \frac{1}{n} \sum_{i=1}^n \log p(x_i \mid \theta) \xrightarrow{n \rightarrow \infty} \mathbb{E}_{x \sim p^*}[\log p(x \mid \theta)]$$

Add and subtract the entropy of p^* (L22 defines it properly):

$$\mathbb{E}_{p^*}[\log p(x \mid \theta)] = \underbrace{\mathbb{E}_{p^*}[\log p^*(x)]}_{\text{fixed: no } \theta} - \underbrace{\text{KL}(p^* \parallel p_\theta)}_{\geq 0, \text{ gap to reality}}$$

Maximizing likelihood $\Leftrightarrow$ minimizing $\text{KL}(p^* \parallel p_\theta)$ — MLE steers the model toward **reality.**

L24 builds KL from scratch and cashes this identity.

Every loss function is a negative log-likelihood in disguise.

Next: **MAP & Conjugate Priors** — what if you have beliefs *before* seeing data?